

Data Architecture Design

1. Context

- Dataset A is an edge-generated event stream (~10,000 events/second) from video-camera object-detection sensors.
- Dataset B is a small, static reference table mapping `geographical_location_oid` to a human-readable location name.
- Objective: dashboard that shows the join of A and B, without duplicates, where results are visible as soon as an event is published, despite duplicate events occurring upstream (retries etc.).

We are solving a streaming, low-latency, exactly-once-effect (not necessarily exactly-once-delivery) problem with a reference-data join as part of the process.

2. Clarifications

Verify the following with the PM and the End User(s)

1. Latency Definition

What does "as soon as published" entail? Is it sub-second required, or a few seconds acceptable? This informs if I can perform batching of event processing, thus determining if the architecture requires true streaming vs micro-batching.

2. Duplicates

1. What exactly will the duplicate events look like? Are they retries of a detection id, or could the same physical event be re-sent with a new detection id. Understanding this addresses how to resolve the dedup part of the data pipeline, as it can either be solved by treating detection ids as idempotent, or may require additional logic on how to combine/dedup entries.
2. How long can a duplicate arrive after the original event, on the order of seconds, minutes, or hours? This determines how large the dedup store needs to be.

3. Dashboard

Is the dashboard a live "board" where we push updates, or is it a tool that queries/pulls the latest data on demand? This determines if the design should be structured around a push-based layer (i.e. WebSocket), or a fast queryable store.

4. Dataset B

Is Dataset B guaranteed to be fully static? How should events which do not have a matching location be handled? Should they be dropped, joined with a placeholder key, or merged? This may have security implications - as for example certain staff should only have access to feeds from certain named locations.

5. Data Volume and Retention

1. Is there a projection for how, if at all, event throughput will scale? Is this a fixed ceiling of 10,000 events/second, or is 10,000 events/second the baseline?
2. Is there any requirement on data retention based on compliance checks or replay requirements?

6. SLA + Infra

Confirm what the team's skillset, availability, CapEx, and OpEx are. Do we have existing infrastructure which we can leverage, or do we need to deploy new resources? Does our cloud service provider have limitations that we need to work around? Are we restricted to on Prem?

7. Schema Evolution

Will there be any changes to the schema of data we ingest? For example are there plans to introduce improved detection attributes such as confidence score?

3. Assumptions

Based on the above questions and additional factors, the assumptions made in the design are:

1. Sub-second-to-a-few-seconds latency is acceptable (near-real-time, not hard real-time).
1. Duplicates are retries of the same detection_oid within a bounded window (within seconds and minutes, not days).
1. The dashboard is a live/push-style view that refreshes continuously.
1. Dataset B changes rarely (on the order of weeks/months), so it can be cached in full at the stream-processing layer and refreshed periodically rather than looked up per-event. Further, any missing locations are dropped due to security requirements.
1. Cloud-agnostic managed services are acceptable; examples below use AWS-style naming but the pattern maps directly onto GCP/Azure/ on-prem Kafka+Flink equivalents.
1. Assume the team already has experience with PySpark/Spark.

4. Architecture

```
Edge cameras --> Ingestion (Kafka / Kinesis) --> Stream processor
    (Spark Structured Streaming / Flink)
    - dedup by detection_oid (stateful, watermarked)
    - broadcast-join against Dataset B (cached reference table)
    --> Serving store (low-latency, queryable)
    --> Dashboard (push or fast-poll)

Locations Mapping -> Dataset B (static reference) --> Object storage / small Online Transaction
Processing (OLTP) table
    --> periodically refreshed broadcast/lookup cache in the stream processor
```

4.1. Ingest

For ingest, we deploy Kafka as it is a replayable, ordered log. It will handle the edge camera input and pass it onto the stream processing.

This decouples the source of the data from the processing layer, allowing backfill of data if necessary, smoothing data input, and also handles the at-least-once delivery for the processing to address.

4.2. Processing

For the stream processing and dedup, leverage Spark Structured Streaming to consume from Kafka. Dedup is handled with stateful, watermarked reduction on detection id. Keeping a bounded state store of recently-seen detection id values allows handling of the retry window without overflowing and eating up excessive memory.

4.3. Join with Location Name Map

We have already proven in this exercise the effectiveness of broadcast hash join. A similar design is leveraged here. Deploy Dataset B to a cheap durable storage such as S3. The stream processor will load the table once at startup, and use it as a broadcast/in-memory cache. Updates to the table can be handled by a scheduler that refreshes the table in memory every few days or weeks. In this join, any location ids without the corresponding labels are dropped silently due to the security assumption in the design.

4.4. Serving Layer

With the stream's processing completed, the data is then written to Apache Druid or Pinot to enable a low-latency, queryable serve store. This allows for sub-second aggregate queries.

At the same time, the raw and joined events should be archived to object storage (S3, organized by date/location) as Parquet. This allows the historical data to be re-run to perform historical analysis, audits, and/or model evaluation.

4.5 Dashboard

The deployed dashboard subscribes to a pub/sub WebSocket gateway fed by the serving store change feed to ensure constant push updates.

5. Overall Justifications

1. The deployed Dashboard shows A join B, without duplicates
 - > stateful, watermarked dedup on detection_oid before the join; broadcast join against the (rarely-changing) Dataset B cache.
2. Results available as soon as published:
 - > true streaming consumption from Kafka with a broadcast-cached reference table (no per-event remote lookup latency), landing in a sub-second query-capable serving store.
3. Duplicate events from retries:
 - > handled by the watermarked dedup state store, sized according to a retry window, bounding memory while still catching realistic retry delays.